

Counting Sort

Last Updated : 20 Aug, 2025



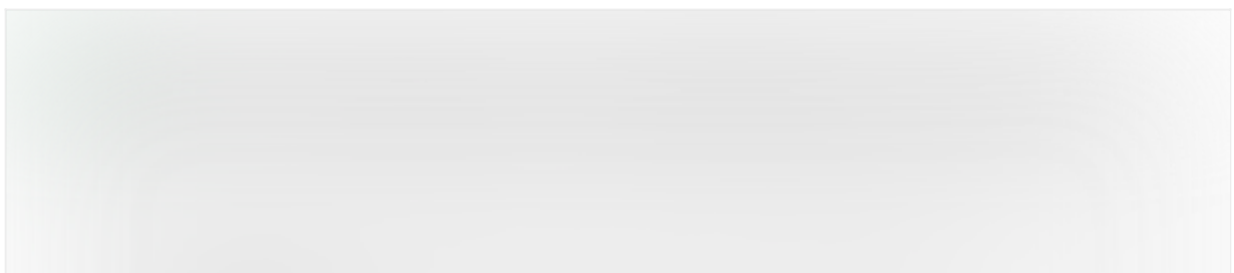
Counting Sort is a **non-comparison-based** sorting algorithm. It is particularly efficient when the **range of input values is small** compared to the number of elements to be sorted.

- The basic idea behind **Counting Sort** is to count the **frequency** of each distinct element in the input array and use that information to place the elements in their correct sorted positions.
- For example, for input [1, 4, 3, 2, 2, 1], the output should be [1, 1, 2, 2, 3, 4]. The important thing to notice is that the range of input elements is small and comparable to the size of the array.
- If the **max element is of order** more than **$n \log n$** where n is size of the array, then we can better sort the array using a standard comparison based sorting algorithm like Merge Sort.

Counting Sort Algorithm

- Declare an **auxiliary array** `countArray[]` of size **$\max(\text{inputArray[]})+1$** and initialize it with 0. **frequency array by index = repetition of elem**
- Traverse array `inputArray[]` and map each element of `inputArray[]` as an index of `countArray[]` array, i.e., execute `countArray[inputArray[i]]++` for $0 \leq i < N$.
- Calculate the prefix sum at every index of array `inputArray[]`.
- Create an array `outputArray[]` of size N .
- Traverse array `inputArray[]` from end and update `outputArray[ countArray[ inputArray[i] ] - 1 ] = inputArray[i]`. Also, update `countArray[ inputArray[i] ] = countArray[ inputArray[i] ] - 1`.

Working of Counting Sort



01
Step

Find out the maximum element from the given array.

	0	1	2	3	4	5	6	7	Max
arr[] =	2	5	3	0	2	3	0	3	5

02
Step

Initialize a cntArr[] of length max+1 with all elements as 0. This array will be used for storing the occurrences of the elements of the input array.

	0	1	2	3	4	5
cntArr[] =	0	0	0	0	0	0

03
Step

In the cntArr[], store the count of each unique element of the input array at their respective indices.

	0	1	2	3	4	5
cntArr[] =	2	0	2	3	0	1

04
Step

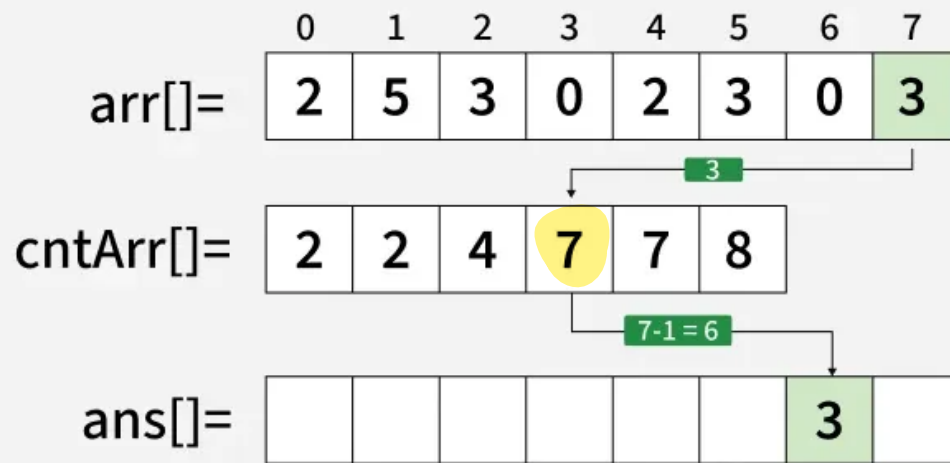
Store the cumulative sum or prefix sum of the elements of the cntArr[] by doing $\text{cntArr}[i] = \text{cntArr}[i - 1] + \text{cntArr}[i]$.

	0	1	2	3	4	5
cntArr[] =	2	2	4	7	7	8

$i = N \rightarrow i =$

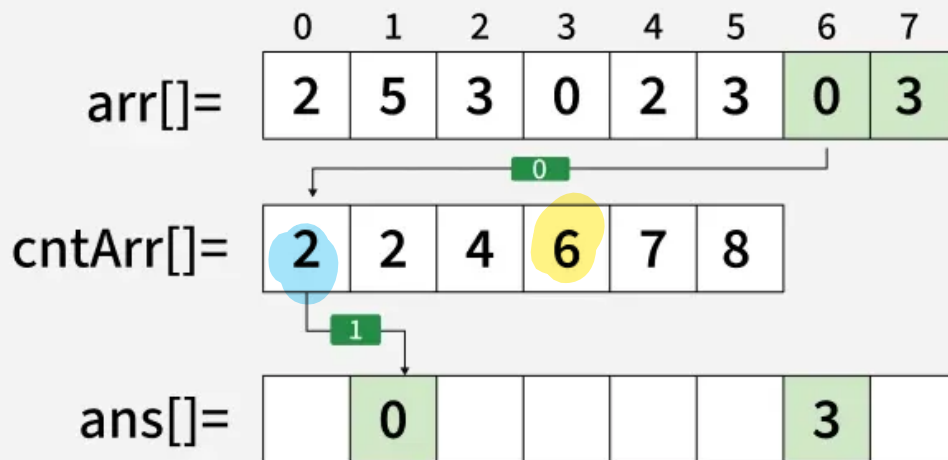
05
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[i]] - 1] = \text{arr}[i]$ and decrement $\text{cntArr}[\text{arr}[i]]$.
Traverse the input array in reverse to maintain the order of equal elements, ensuring the sort remains stable.



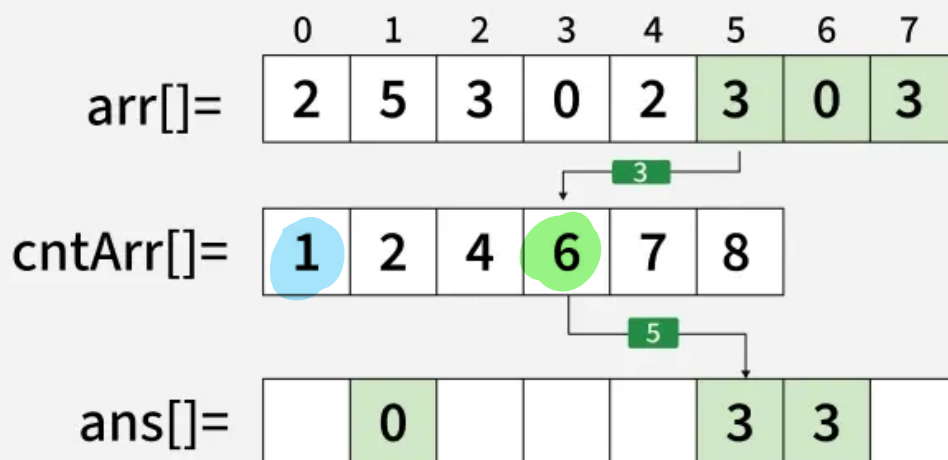
06
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[6]] - 1] = \text{arr}[6]$
Also, update $\text{cntArr}[\text{arr}[6]] = \text{cntArr}[\text{arr}[6]] - 1$



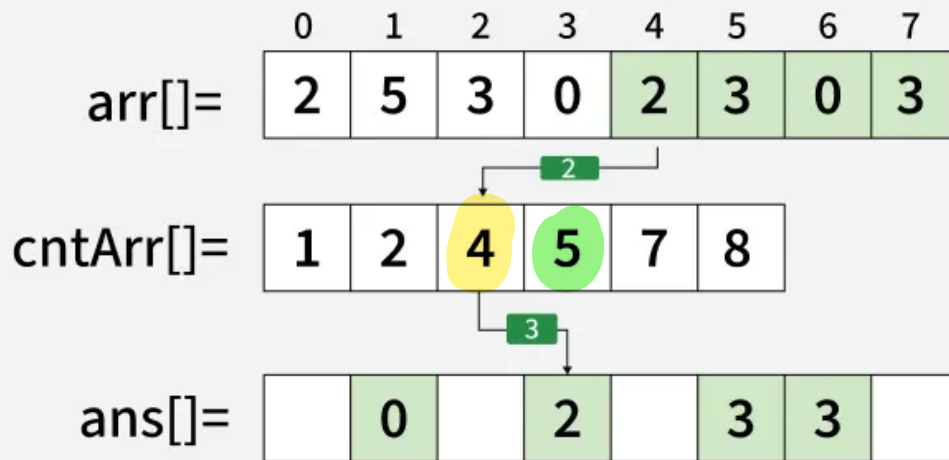
07
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[5]] - 1] = \text{arr}[5]$
Also, update $\text{cntArr}[\text{arr}[5]] = \text{cntArr}[\text{arr}[5]] - 1$



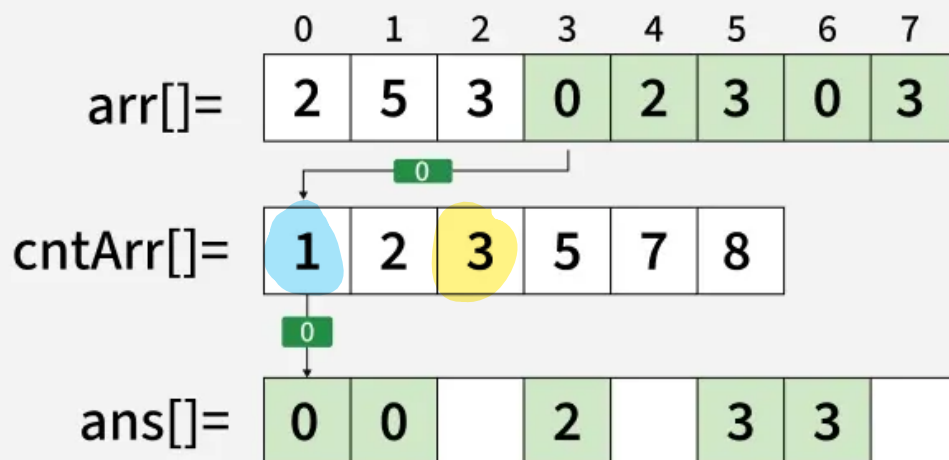
08
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[4]] - 1] = \text{arr}[4]$
Also, update $\text{cntArr}[\text{arr}[4]] = \text{cntArr}[\text{arr}[4]] - 1$



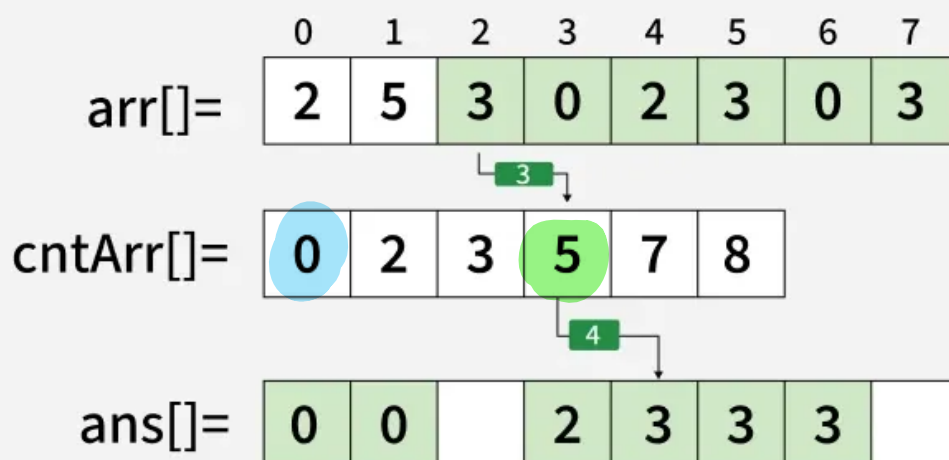
09
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[3]] - 1] = \text{arr}[3]$
Also, update $\text{cntArr}[\text{arr}[3]] = \text{cntArr}[\text{arr}[3]] - 1$



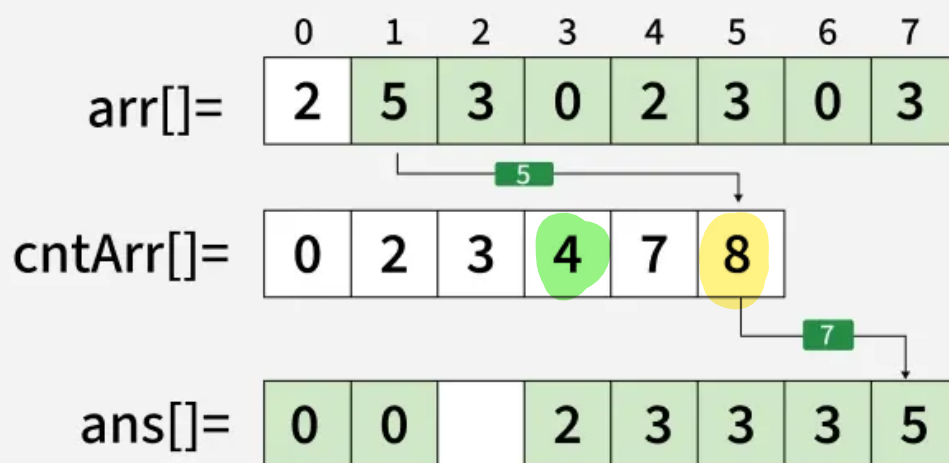
10
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[2]] - 1] = \text{arr}[2]$
Also, update $\text{cntArr}[\text{arr}[2]] = \text{cntArr}[\text{arr}[2]] - 1$



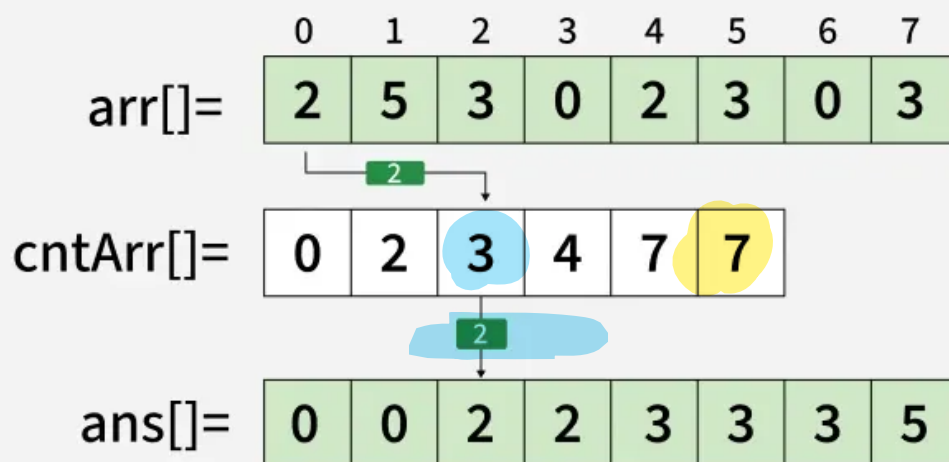
11
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[1]] - 1] = \text{arr}[1]$
Also, update $\text{cntArr}[\text{arr}[1]] = \text{cntArr}[\text{arr}[1]] - 1$



12
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[0]] - 1] = \text{arr}[0]$
Also, update $\text{cntArr}[\text{arr}[0]] = \text{cntArr}[\text{arr}[0]] - 1$



C++

C

Java

Python

C#

JavaScript

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> countsort(vector<int>& arr) {
    int n = arr.size();

    // find the maximum element
    int maxval = 0;
    for (int i = 0; i < n; i++)
        maxval = max(maxval, arr[i]);

    // create and initialize count array
    vector<int> count(maxval + 1, 0);

    // count frequency of each element
    for (int i = 0; i < n; i++)
        count[arr[i]]++;

    // compute prefix sum
    for (int i = 1; i <= maxval; i++)
        count[i] += count[i - 1];

    // build output array
    vector<int> ans(n);
    for (int i = n - 1; i >= 0; i--) {
        ans[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    return ans;
}
```

```

}

int main() {
    vector<int> arr = {2,5,3,0,2,3,0,3};
    vector<int> sorted = countsort(arr);

    for (int x : sorted)
        cout << x << " ";

    return 0;
}

```

Output

```
0 0 2 2 3 3 3 5
```

Complexity Analysis of Counting Sort:

- **Time Complexity:** $O(N+M)$, where **N** and **M** are the size of **inputArray[]** and **countArray[]** respectively.
 - Worst-case: $O(N+M)$.
 - Average-case: $O(N+M)$.
 - Best-case: $O(N+M)$.
- **Auxiliary Space:** $O(N+M)$, where **N** and **M** are the space taken by **outputArray[]** and **countArray[]** respectively.

Advantage of Counting Sort:

- Counting sort generally performs faster than all comparison-based sorting algorithms, such as merge sort and quicksort, if the range of input is of the order of the number of input.
- Counting sort is easy to code
- Counting sort is a **stable algorithm**.

Disadvantage of Counting Sort:

- Counting sort **doesn't work on decimal values**.
- Counting sort is inefficient **if the range of values to be sorted is very large**.

- Counting sort is not an **In-place sorting** algorithm, It uses extra space for sorting the array elements.

Applications of Counting Sort:

- It is a commonly used algorithm for the cases where we have limited range items. For example, sort students by grades, sort a events by time, days, months, years, etc
- It is used as a subroutine in [Radix Sort](#)
- The idea of counting sort is used in [Bucket Sort](#) to divide elements into different buckets.

Counting Sort

 Comment

More info ▼

Advertise with us

Similar Reads

Basics & Prerequisites



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information